

1) What is dependency injection in angular and how does it help in applications development?

a) Dependency Injection is a design pattern used to supply a class with its required dependencies from the outside rather than creating them internally. In angular, DI is a built-in feature that allows services or objects to be injected into components, directives, pipes.

benefits:-

- Decoupling
- Reusability
- Testability
- Maintainability.

2) How do you create and register a service in angular? Explain the role of @Injectable() and providers.

a) 1) Create a service:-

ng g s my-service

2) Inside the service file -

import { Injectable } from '@angular/core';

@Injectable({
 providedIn: 'root',

3)

export class MyService {
 constructor() {}

}

@Injectable() role:-

- Marks a class as available for DI
- Allows Angular to inject dependencies into this service of required

Rule of provided in :-

- specifies where the service should be provided (root/module/component)
- 'root' means a singleton is available app-wide

3) what is the difference between providing a service in root, a module or a component?

where provided	Scope	Instance Behavior
root	Application-wide	singleton shared everywhere
Specific module	that module and its components	new instance for module
component	that component and its children	unique instance per component tree

4) Explain the concept of hierarchical injectors in angular with an example?

- angular builds a hierarchical tree of injectors that mirrors the component tree

how it works :-

- the root injector provides singleton services

- components can have their own injectors if they declare providers

- Angular searches upward in the hierarchy to find the dependency

Ex:-

```
@component({
  selector: 'parent',
  providers: [myService],
```

3)

```
export class parentComponent {}
```

```
@component({
  selector: 'child',
```

3)

```
export class childComponent {
```

```
  constructor(private myService: myService) {}
```

3

5) what is the purpose of the @Inject() decorator in Angular DI? when would you use it instead of constructor injection alone?

→ @Inject() allows you to explicitly specify the tokens used for injection.

use cases:-

→ when injecting a value or token that isn't a class

→ when using InjectionToken for interfaces or configurations.

```
import { inject, InjectionToken } from '@angular/core'
```

```
export const API_URL = new InjectionToken<string>
```

```
('apiURL');
```

```
constructor(@Inject(API_URL) private apiUrl:
```

```
String) {}
```

6) what is the difference between @optional() and @self() in Angular dependency injection?

Decorator	Purpose	Behaviour
@optional()	Tells Angular to inject null if dependency isn't found	prevents DI error
@self()	Tells Angular to look only in the local injector	doesn't search up the hierarchy

use case:-

@optional()

constructor (@optional() private logger?: LoggerService) {}

@self():-

constructor (@self() private formControl: NgControl) {}

7) what are the differences between useClass, useValue, useFactory, useExisting in service providers?

Type	Description	Example
useClass	provide a class for the token	{ provide: Logger, useClass: FileLogger }
useValue	provide a value or object	{ provide: CONFIG, useValue: { api: '...' } }
useFactory	provide based on a factory function	{ provide: oldService, useFactory: () => new MyService() }

use Existing	Alias to another token	if provide, old service, use Existing, New service
--------------	------------------------	--

8) How can you override a service provided in the root injector at a component level? what are the implications?

→ you override it by adding the service to the providers array of the component.

@component({
providers: [myService],

3)

Export class myComponent @3

implications:-

→ The component and its childrens get a new instance

→ If not handled properly, this can break singletons assumptions

9) What is the role of angulars injector class and how can it be used to resolve dependencies manually at runtime?

→ Angular injector is the runtime DI container that holds provider definitions and can dynamically resolve dependencies.

Use Case:-

→ When you don't know the dependency at compile time

→ lazy-loaded modules

→ plugin architectures.

import { Injectable } from '@angular/core';

constructor(private injector: Injector) {}

ngOnInit(): void {

const myService = this.injector.get(MyService);

myService.doSomething();

}